

Name: Shardul Narvekar

Roll No.: UAI05BT223414

Division: B

Subject: Deep Learning

Q9.d) Implementing XOR gate, using a single perceptron

First understand the input data

x1 x2 y 0 0 0 1 1 1 0 1 1 1 0

We will represent this in the form of matrix - Rank 2 Tensor Thus we will use the numpy library

```
import numpy as np
#we import the required library

input_data=np.array([[0,0],[0,1],[1,0],[1,1]])
#matrix representation

input_data
array([[0, 0],
       [0, 1],
       [1, 0],
       [1, 1]])

input_data.ndim #its rank
2

input_data.shape
(4, 2)

output_data=np.array([0,1,1,0]) #save the output data

output_data
array([0, 1, 1, 0])

output_data.ndim
1

output_data.shape
(4, )

#now we have to define, what are the additional parameters which are
required
```

$z = x_1w_1 + x_2w_2 + b$

#we randomly initialize the weight, e.g. for the 1st epoch, we initialize as 0

#we can define it individually, but going ahead its not a good practice

`w=[w1,w2]`

#thus we do like above

`weights=np.zeros(2)`

`weights`

`array([0., 0.])`

#above is a method to assign the weights as zero

`weights.ndim`

`1`

`weights.shape`

`(2,)`

#bias is a scalar quantity

`bias = 0`

#the no.of times the perceptron will have a look at the entire dataset is called an epoch

`epoch=10`

#Learning Rate (eta)

`eta=0.1`

#now we define the activation function

```
def activation(z):  
    if z>0:  
        return 1  
    else:  
        return 0
```

`len(input_data)`

`4`

```
for i in range (epoch):  
    print(f'Epoch:{i}')  
    for j in range(len(input_data)):  
        print(input_data[j])
```

Epoch:0

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:1

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:2

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:3

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:4

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:5

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:6

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:7

[0 0]
[0 1]
[1 0]
[1 1]

Epoch:8

[0 0]
[0 1]
[1 0]
[1 1]

```

Epoch:9
[0 0]
[0 1]
[1 0]
[1 1]

for i in range (epoch):
    print(f'Epoch:{i}')
    print()
    for j in range(len(input_data)):
        x=input_data[j]
        yactual=output_data[j]
        z=np.dot(x,weights)+bias
        ypred = activation(z)
        error=yactual-ypred
        # print(error)
        #there is an error in the for iteration, thus we update the
weight
        #refer the notebook
        weights=weights+(eta*error*x)
        bias=bias+eta*error
        #above is the formula to update the bias
        #note: the weights are getting updated after the 1st Epoch

        print("Actual Value: ",yactual)
        print("Predicted Value: ",ypred)
        print("Error: ",error)
        print()
    print()
    print()

```

Epoch:0

```

Actual Value:  0
Predicted Value:  0
Error:  0

```

```

Actual Value:  1
Predicted Value:  0
Error:  1

```

```

Actual Value:  1
Predicted Value:  1
Error:  0

```

```

Actual Value:  0
Predicted Value:  1
Error:  -1

```

Epoch:1

Actual Value: 0
Predicted Value: 0
Error: 0

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:2

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:3

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:4

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:5

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:6

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:7

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 0
Predicted Value: 1
Error: -1

Epoch:8

Actual Value: 0
Predicted Value: 1
Error: -1

Actual Value: 1
Predicted Value: 0
Error: 1

Actual Value: 1
Predicted Value: 0

Error: 1

Actual Value: 0

Predicted Value: 1

Error: -1

Epoch:9

Actual Value: 0

Predicted Value: 1

Error: -1

Actual Value: 1

Predicted Value: 0

Error: 1

Actual Value: 1

Predicted Value: 0

Error: 1

Actual Value: 0

Predicted Value: 1

Error: -1

weights

array([-0.1, 0.])

print(bias)

0.1